

Documentazione progetto Ingegneria del Software

Software per la gestione di pazienti diabetici

Toffali Tommaso - VR 500800

August 2026

Contents

1	Requisiti e Use Case	3
1.1	Requisiti	3
1.2	Casi d'uso paziente	3
1.2.1	Inserimento o modifica rilevazioni	4
1.2.2	Inserimento assunzione farmaco	5
1.3	Casi d'uso medico	6
1.3.1	Visualizzazione dei dati del paziente	6
1.3.2	Modifica dei dati del paziente	6
1.3.3	Modifica o aggiunta di terapie	6
1.4	Ricezione delle notifiche	9
1.5	Casi d'uso admin	11
1.5.1	Gestione degli utenti	11
1.5.2	Gestione delle richieste di registrazione	11
1.5.3	Visualizzazione delle modifiche dei medici	12
1.6	Activity diagram	12
1.6.1	Registrazione di un utente	12
1.6.2	Attività del paziente	13
1.6.3	Attività del medico	14
1.6.4	Attività dell'amministratore	16
2	Sviluppo	18
2.1	Processo di sviluppo	18
2.2	Progettazione e pattern utilizzati	19
2.3	Comunicazione e gestione delle richieste	20
2.4	Implementazione e design pattern utilizzati	21
2.4.1	Generalità	21
2.4.2	Autenticazione e autorizzazione	22
2.4.3	Model: repository e service layer	22
3	Test e validazione	24
3.1	Revisione del codice	24
3.2	Test dei service e dei repository	24
3.3	Test dei controller	25
3.4	Test sicurezza	25
3.5	Test manuale dello sviluppatore	25
3.6	Test utenti generici	26

1 Requisiti e Use Case

1.1 Requisiti

Il software sviluppato è un sistema di telemedicina di un servizio clinico per la gestione di pazienti diabetici. Il sistema permette l'interazione di due attori principali: il diabetologo e il paziente. Questi creano un nuovo utente che, dopo essere stato verificato da un admin, finiscono di creare il loro profilo con i dati relativi al ruolo scelto.

Una volta autenticato ogni utente verrà reindirizzato alla propria pagina home, nella quale visualizzerà le azioni e informazioni relative al proprio ruolo. Di seguito è presente un diagramma che rappresenta i casi d'uso per ogni utente dopo la corretta autenticazione.

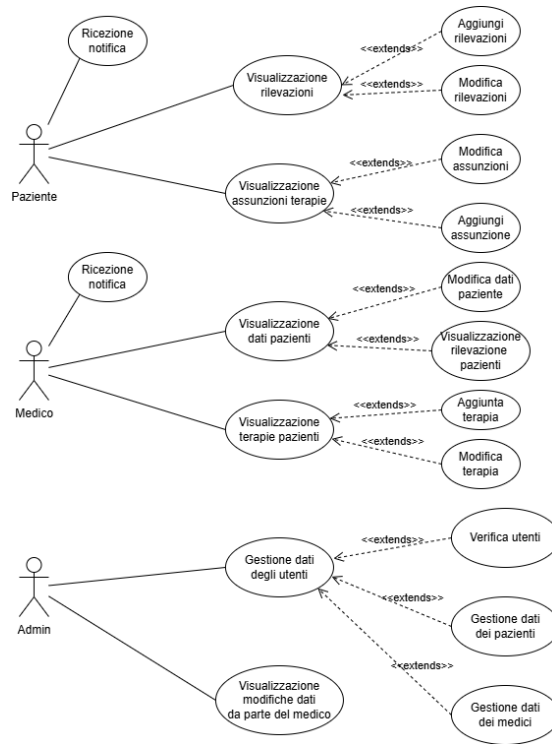


Figure 1: Casi d'uso per ruolo

1.2 Casi d'uso paziente

Dopo che il paziente si è autenticato ed entra nella sua pagina home dove visualizza tutte le azioni a lui disponibili.

1.2.1 Inserimento o modifica rilevazioni

Una delle pagine disponibili per il paziente è quella di visualizzazione delle rilevazioni passate che sono state effettuate, con relativa possibilità di modifica di queste, e la possibilità di registrare i dati di una nuova rilevazione.

Attore: Paziente

Precondizioni: il paziente deve essersi autenticato correttamente

Passi:

1. Il paziente dalla home page accede alla pagina di visualizzazione e modifica delle rilevazioni.
2. Il paziente può:
 - inserire i dati per una nuova rilevazione: livelli di glicemia, se è stata effettuata prima o dopo un pasto, data e ora.
 - modificare i dati di una rilevazione passata, tranne la data e ora.
 - Può inserire eventuali sintomi.
3. Il paziente conferma l'inserimento della rilevazione.

Postcondizioni: La rilevazione viene salvata/aggiornata e se i livelli di glicemia sono al di fuori del normale viene inviata una notifica al medico di riferimento del paziente.

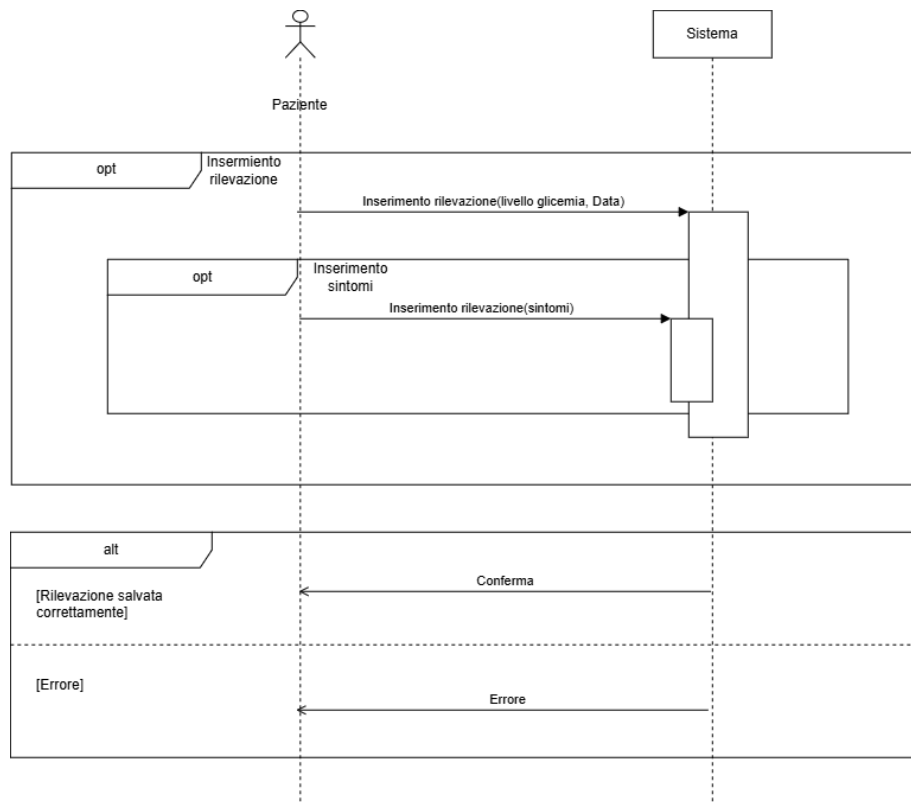


Figure 2: Sequence diagram per inserimento rilevazione paziente

1.2.2 Inserimento assunzione farmaco

Un'altra pagina disponibile per il paziente riguarda la visualizzazione delle terapie che gli sono state prescritte dal medico, con relativa possibilità di aggiungere una nuova assunzione di un farmaco tra quelli che ha presenti nelle terapie.

Attore: Paziente

Precondizioni: il paziente deve essersi autenticato correttamente

Passi:

1. Il paziente dalla home page accede alla pagina di visualizzazione delle terapie e assunzioni di farmaci.
2. Il paziente preme il bottone per inserimento di una nuova assunzione:
 - Inserisce la medicina assunta, la quantità presa, la data e ora.
3. Il paziente conferma l'inserimento dell'assunzione.

Postcondizioni: L'assunzione del farmaco viene salvata nel sistema, questo controlla che l'assunzione sia regolare con la terapie prescritta dal medico e in caso negativo notifica il medico di riferimento.

1.3 Casi d'uso medico

Dopo che il medico si è autenticato ed entra nella sua pagina home dove visualizza tutte le azioni a lui disponibili.

1.3.1 Visualizzazione dei dati del paziente

Attore: Medico

Precondizioni: Il medico deve essere autenticato correttamente

Passi:

1. Il medico accede alla sua area riservata
2. Il medico puo' accedere alla lista dei pazienti
3. Il medico puo' visualizzare i dati del paziente e lo storico delle rilevazioni

Postcondizioni:nessuna

1.3.2 Modifica dei dati del paziente

Attore: Medico

Precondizioni: Il medico deve essere autenticato correttamente

Passi:

1. Il medico dalla sua pagina home accede alla pagina di visualizzazione dati dei pazienti.
2. Il medico visualizza una lista dei suoi pazienti con i suoi dati principali.
3. Il medico seleziona l'azione di modifica che si trova a fianco di ogni riga di un paziente.
4. Il medico può modificare i dati relativi a fattori di rischio del paziente.
5. Il medico conferma le modifiche dei dati del paziente.

Postcondizioni: Il sistema modifica i dati del paziente e viene salvata nella lista delle tracce.

1.3.3 Modifica o aggiunta di terapie

Attore: Medico

Precondizioni: Il medico deve essere autenticato correttamente

Passi:

1. Il medico dalla sua pagina home accede alla pagina di visualizzazione delle terapie.
2. Il medico visualizza una lista delle terapie che sono collegate ai suoi pazienti.
3. Il medico può:
 - modificare una terapia già assegnata.
 - aggiungere una nuova terapia ad un paziente.
4. Il medico conferma le modifiche che ha effettuato.

Postcondizioni: Il sistema modifica i dati della terapia che è stata aggiunta/modificata.

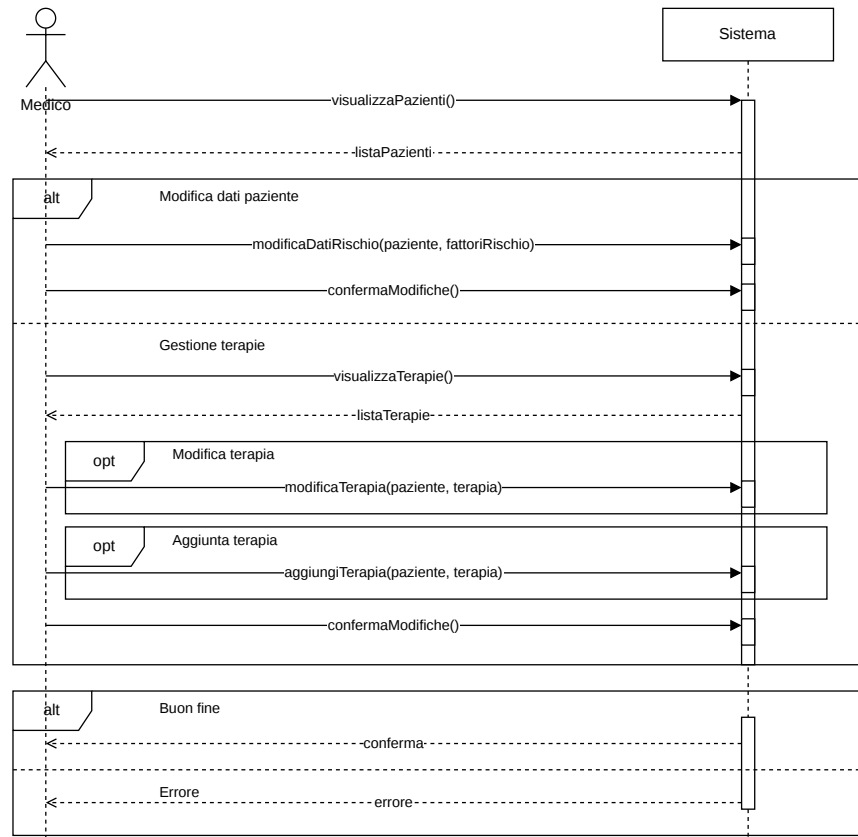


Figure 3: Sequence Diagram del medico

1.4 Ricezione delle notifiche

Il paziente e il medico possono ricevere delle notifiche dal sistema, se il paziente si dimentica di assumere i farmaci il sistema manda una notifica per ricordarglielo, oppure se le soglie di glicemia o sono leggermente più alte/basse o sono gravemente più alte/basse e in base a questo cambia anche il tipo di notifica.

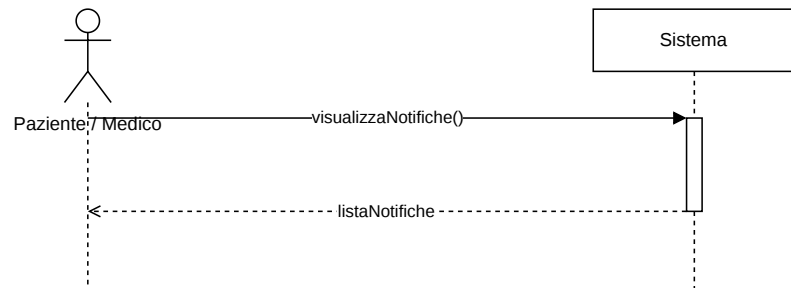
Attore: Paziente o Medico

Precondizioni: Il paziente o medico devono essere autenticati

Passi:

1. Il paziente o medico accede alla sua home page
2. Il paziente o medico visualizza la campanella delle notifiche
3. Il paziente o medico possono visualizzare:
 - Notifiche lette
 - Notifiche da leggere

Postcondizioni: Se viene visualizzata una notifica, questa viene marcata come letta



Il sistema invia delle notifiche ai seguenti attori:

- **Paziente:** avvisa il paziente che ha dimenticato di assumere il farmaco
- **Al medico di riferimento del paziente:** per avvisare che il paziente non segue la terapia da più di tre giorni
- **A tutti i medici:** per segnalare i pazienti che registrano livelli di glicemia sopra le soglie

1.5 Casi d'uso admin

L'amministratore del sistema gestisce gli utenti del sistema, si occupa di modificare/cancellare account di pazienti e medici. La registrazione viene effettuata tramite un form dove viene specificato il ruolo, prima che un utente possa utilizzare le funzioni relative al suo ruolo nel sistema l'amministratore deve autorizzare la richiesta di creazione dell'account da parte dell'utente

1.5.1 Gestione degli utenti

Attore: Admin

Precondizioni: L'admin deve essere autenticato correttamente

Passi:

1. L'admin dalla sua pagina home accede alla pagina di gestione degli utenti
2. L'admin visualizza la lista degli utenti
3. L'admin può:
 - Modificare un utente
 - Eliminare un utente

Postcondizione: L'utente è stato modificato o eliminato.

1.5.2 Gestione delle richieste di registrazione

Attore: Admin

Precondizioni: L'admin deve essere autenticato correttamente

Passi:

1. L'admin dalla sua pagina home accede alla pagina di gestione degli utenti
2. L'admin visualizza la lista degli utenti
3. L'admin può:

- Accettare la richiesta di registrazione di un utente
- Rifiutare la richiesta di registrazione di un utente

Postcondizione: Se accettata l'utente viene verificato e può procedere con il completamento dei dati, altrimenti niente.

1.5.3 Visualizzazione delle modifiche dei medici

Il sistema tiene traccia delle modifiche che i medici effettuano sui pazienti

Attore: Admin

Precondizioni: L'admin deve essere autenticato correttamente

Passi:

1. L'admin dalla sua pagina home accede alla pagina di visualizzazione delle modifiche da parte dei medici
2. L'admin visualizza la lista delle modifiche effettuate

Postcondizione: Nessuna

1.6 Activity diagram

1.6.1 Registrazione di un utente

Solamente pazienti e medici posso registrarsi al sistema, la loro registrazione consente nella creazione di un utente con il proprio ruolo, una volta verificato dall'admin questo può terminare la registrazione completando i dati per il ruolo che ha scelto.

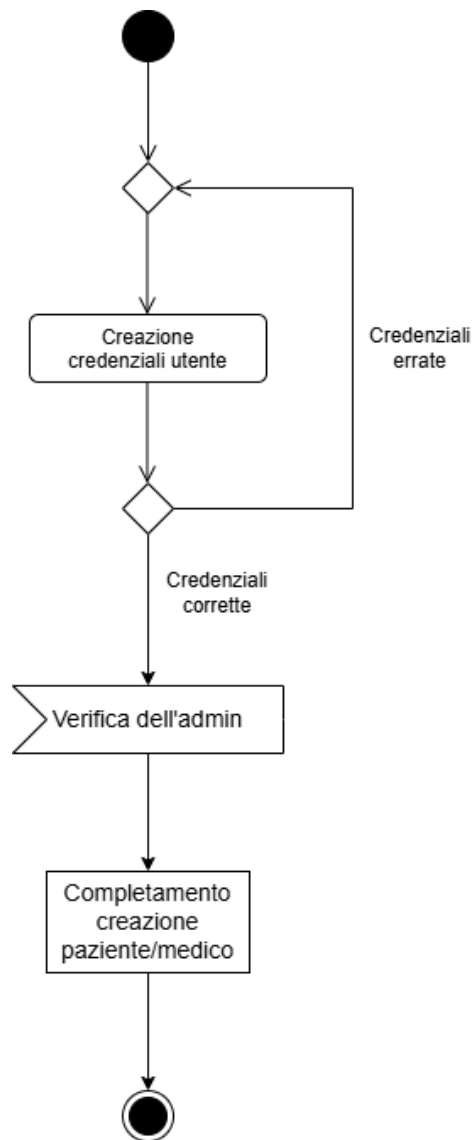


Figure 5: Registrazione da parte di un utente

1.6.2 Attività del paziente

Di seguito è presente un activity diagram per un paziente in seguito alla corretta autenticazione al sistema.

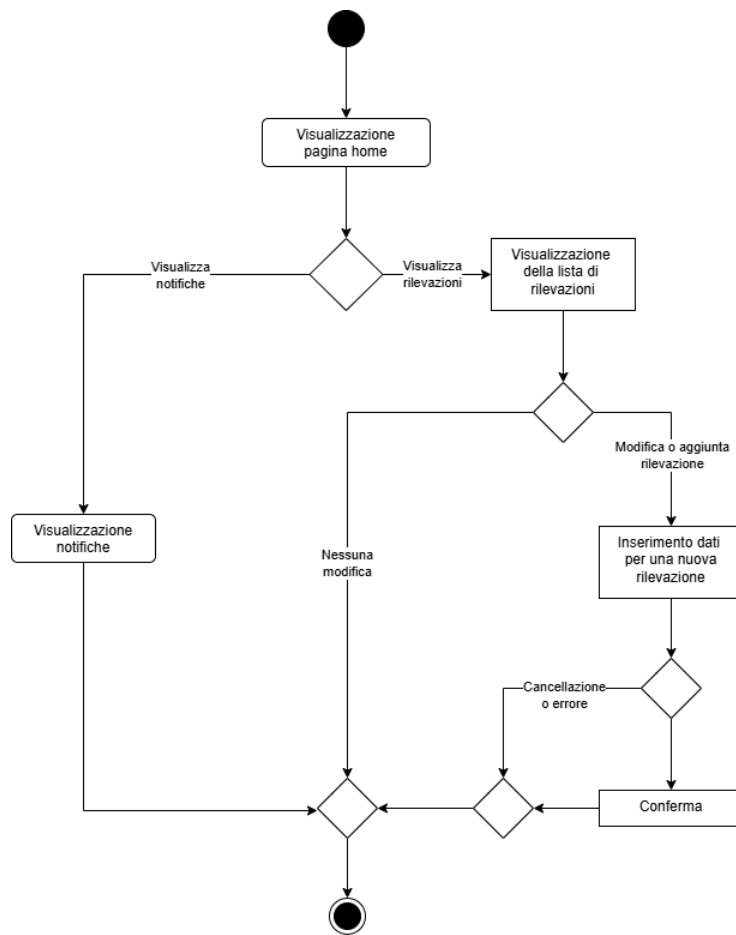


Figure 6: Activity diagram di un Paziente

1.6.3 Attività del medico

Di seguito è presente un activity diagram per un medico in seguito alla corretta autenticazione al sistema.

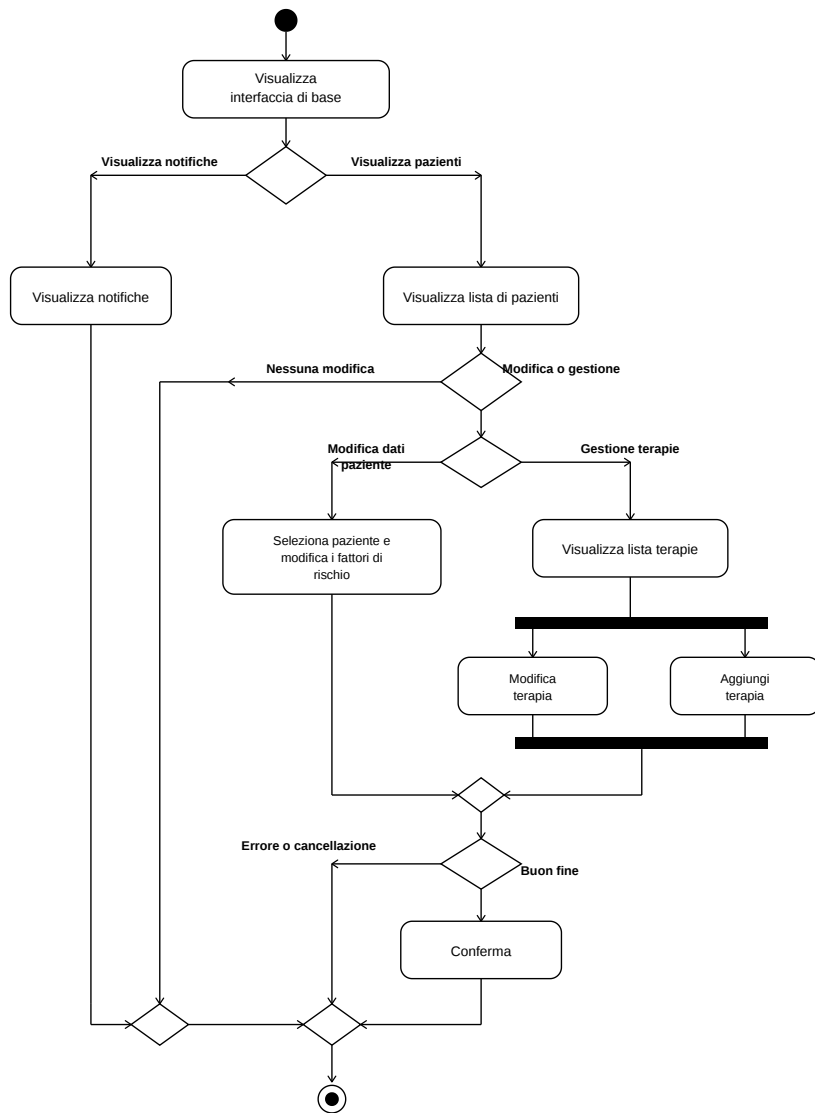


Figure 7: Activity Diagram del medico

1.6.4 Attività dell'amministratore

Di seguito è presente un activity diagram per un admin in seguito alla corretta autenticazione al sistema.

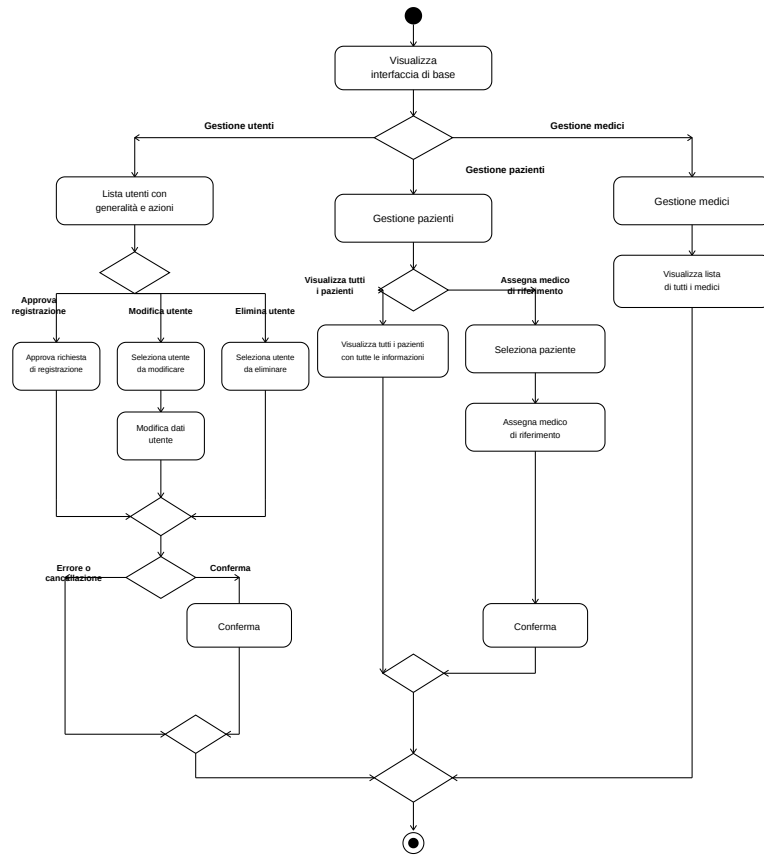


Figure 8: Activity Diagram dell'Admin

2 Sviluppo

2.1 Processo di sviluppo

Il processo di sviluppo del software è stato condotto individualmente, seguendo un processo **iterativo e incrementale**, ispirato ai principi *Agile* ma semplificato nello sviluppo, organizzato e gestito tramite *GitHub* e ad un flusso di lavoro Git basato su commit frequenti e granularità fine.

La pianificazione del progetto prevede rilasci incrementali, ciascuno corrispondente ad un macro-requisito individuato nella fase di analisi (si veda la sezione 1): partendo dal sistema di autenticazione autorizzazione, lo sviluppo è proceduto in modo verticale, implementando di volta in volta la catena di *Model* $\rightarrow$ *Repository* $\rightarrow$ *Service* $\rightarrow$ *Controller* lato backend, e a seguire il corrispondente componente *Angular* lato frontend, in modo da poter validare end-to-end ogni funzionalità appena completata, prima di passare alla pagina successiva. Questo approccio ha permesso di individuare in maniera tempestiva incoerenze tra il contratto esposto dalle API REST e le aspettative del client, andando a ridurre il rischio di dover ricontrollare ampie porzioni di codice in fase avanzate di sviluppo.

L'ordine di sviluppo delle funzionalità principali è il seguente:

1. **Autenticazione:** login, registrazione, verifica da parte dell'amministratore e configurazione di sicurezza basata sui token JWT
2. **Rilevazioni:** gestione delle rilevazioni glicemiche giornaliere del paziente
3. **Terapie:** gestione delle terapie prescritte dal medico e delle relative assunzioni di farmaci da parte del paziente, incluso il controllo di aderenza alla terapia
4. **Condizioni concomitanti:** la scelta implementativa in questa sezione è ricaduta sul fatto che il paziente potesse inserire delle auto-dichiarazioni di sintomi, patologie o terapie che segue in modo parallelo rispetto a quelle prescritte dal medico e non legate al piano di cura diabetico.
5. **Notifiche:** sistema di notifiche automatiche, sia reattive (per eventi come l'inserimento di una rilevazione fuori dalla soglia) sia proattive (tramite controlli pianificati periodicamente).
6. **Dashboard:** costruiti aggregando dati già esposti dalle API sviluppate nei punti precedenti.

Per la gestione del codice sorgente, come sopraindicato, è stato usato **Git** come sistema di versionamento, con la creazione di due repository, una dedicata al backend ed una dedicata al frontend. Questa scelta è motivata dal fatto di voler disaccoppiare il ciclo di vita e cronologia dei due sottosistemi, che pur comunicando tramite un contratto REST condiviso evolvono con cadenze e logiche

diverse. Come ambienti di sviluppo ho deciso di adottare **IntelliJ IDEA** per la parte di Java/Spring Boot e **Visual Studio Code** per la parte di Angular/TypeScript

2.2 Progettazione e pattern utilizzati

Il sistema è stato progettato secondo un architettura **Client-Server** a due livelli, comunicanti tramite API REST su protocollo HTTP:

- **Client (Single Page Application):** realizzato con **Angular** è responsabile della presentazione dei dati e dell'interazione con l'utente. Il client non contiene logica di business (es. calcolo delle soglie glicemiche, verifiche aderenze terapia), che sono state delegate al server; si occupa invece della validazione sintattica dei form, gestione dello stato locale della UI e orchestrazione delle chiamate HTTP verso il backend.
- **Server:** realizzato con **Spring Boot** espone un insieme di risorse REST che sono organizzate per dominio (rilevazioni glicemiche, terapie, assunzioni farmaci, condizioni concomitanti, notifiche, gestione utenti) ed esso comprende sia la logica di business sia le regole di autorizzazione basate sul ruolo dell'utente autenticato.

All'interno del server è stata adottata un'**architettura a livelli**:

- **model:** Ospita le **JPA Entity** che riletono la struttura delle tabelle relazionali (User, Patient, Medic, Therapy, MedicineIntake, GlycemiaReading, Symptom, Pathology, ConcomitantTherapy, Notification), ciascuna di esse è stata mappata usando le annotazioni *Jakarta Persistence* (@Entity, @ManyToOne, @Column, ecc.);
- **repository:** racchiude le interfacce che estendono la *JpaRepository*, che forniscono l'accesso ai dati tramite query;
- **service:** contiene la logica di business, riguardanti tutte le regole che permettono i vari controlli riguardanti assunzioni, terapie, patologie.
- **controller:** pubblica gli endpoint REST, andando ad affidare ogni operazione al corrispondente service e traducendo il risultato in una risposta HTTP con l'apposito status code.
- **config:** contiene la configurazione di sicurezza (SecurityConfiguration) e il filtro per l'autenticazione JWT (JwtFilter).

Questa implementazione, per quanto non rispecchi in modo del tutto preciso il pattern Model-View-Controller (essendo la View del tutto esterna al server), ne condivide il principio cardine, quindi, la separazione tra la rappresentazione dei dati (Model), l'esposizione delle operazioni disponibili (Controller), e la presentazione all'utente (View, controllata dal client Angular).

A seguire sono rappresentati i diagrammi UML delle classi del Model e del Controller, che rappresentano le entità principali del software e le loro relazioni.

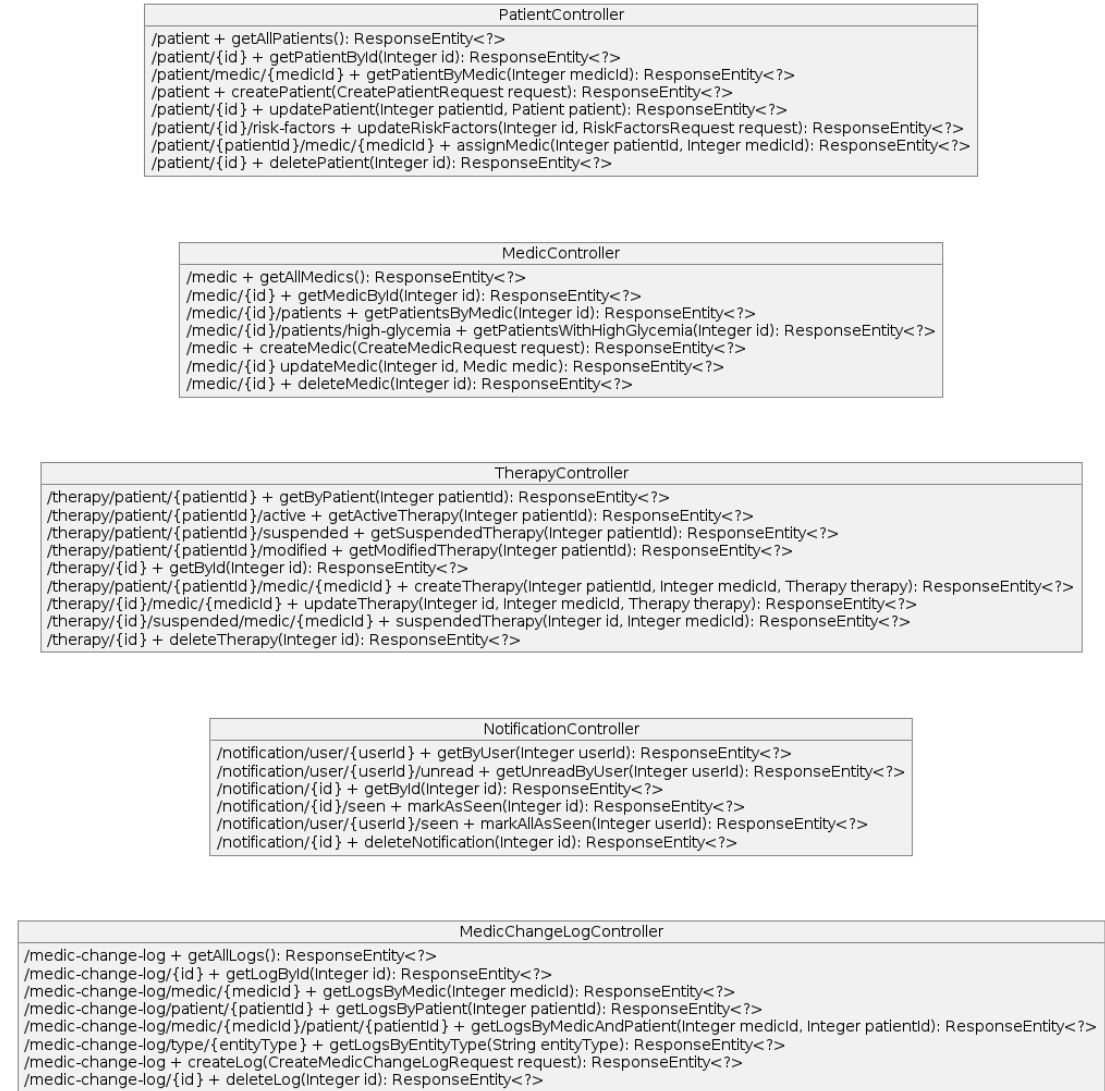


Figure 9: Classi UML dei controller principali

2.3 Comunicazione e gestione delle richieste

La comunicazione tra client e server avviene tramite le richieste HTTP, con il payload che è in formato **JSON**, rispecchiando il paradigma REST, dove ogni risorsa del dominio è caratterizzata da un URI dedicato (es. `/therapy/patient/patientId`),

e le operazioni su di essa sono distinte tramite HTTP usando la **GET** per la lettura, **POST** per la creazione, **PUT** per l'aggiornamento, **PATCH** per fare aggiornamenti parziali (come le chiusure), **DELETE** per la cancellazione.

Lato client, tutte le chiamate HTTP sono gestite da un unico servizio Angular (`HttpClientService`) il quale non interagisce direttamente ma invoca i metodi (es. `nuovaAssunzioneFarmaco`, `getTerapieAttive`, `chiudiCondizione`) al cui interno è presente la costruzione dell'URL e la gestione della richiesta.

L'autenticazione delle richieste avviene tramite un **HTTP Interceptor**, lato angular (`authInterceptor`), che intercetta qualsiasi richiesta che esce e se è presente un token JWT valido, quest'ultimo viene automaticamente allegato all'header *Authorization* in formato *Bearer token*. Questa scelta implementativa mi permette di evitare che la logica di autenticazione debba essere replicata manualmente in ciascuna chiamata.

I dati che riceviamo dal backend (in formato JSON) vengono deserializzati (lato client) nelle istanze di classi *TypeScript* dedicate (es. `MedicineIntake`, `Therapy`, `AppNotification`), dove i costruttori effettuano un'opera di normalizzazione, in particolar modo la conversione delle stringhe di data nel formato **ISO-8601** in istanze *Date* native, andando così a disaccoppiare la rappresentazione di trasporto (JSON) da quella usata internamente nel client.

2.4 Implementazione e design pattern utilizzati

2.4.1 Generalità

Durante lo sviluppo sono stati adottati, diversi pattern di progettazione:

- **Repository/DAO Pattern:** viene implementato usando le interfacce *Spring Data JPA* che estendono **JpaRepository**. Ogni repository assume il compito di *Data Access Object* per la corrispondente entità isolando i dettagli dell'accesso al database relazionale.
- **Wrapper:** applicato, lato backend, ogni classe *Service* funge da facciata al di sopra di uno o più repository, andando ad esporre operazioni di business di livello più alto (es. `checkConsistencyWithTherapy`, `checkGlycemiaAlertForReading`). Lato frontend, come citato poc'anzi, *HttpClientService* assume il compito di facciata che punta all'insieme di risorse RESP che vengono esposte dal backend.
- **Observer pattern:** presente lato backend, dove le relazioni JPA (`@ManyToOne`) fanno in modo che la modifica di un'entità referenziata come ad esempio *Therapy* venga automaticamente riflessa nelle entità correlate al momento del fetch
- **Strategy pattern (applicato all UI):** riguardo la gestione delle condizioni concomitanti è stato realizzato un componente generico (Con-

ditionTable) che determina sia le colonne da visualizzare sia l'endpoint REST da invocare.

2.4.2 Autenticazione e autorizzazione

Per quanto riguarda la gestione dell'autenticazione e dell'autorizzazione è stata usata la libreria **Spring Security**, in modo coerente con la natura REST: il server non mantiene la sessione lato backend ed ogni richiesta viene autenticata tramite un token **JWT (JSON Web Token)**

Le principali classi coinvolte nella configurazione della sicurezza sono:

- **JwtFilter**: si tratta di un filtro Servlet, il cui compito è estrarre il token JWT dalla richiesta in ingresso e verificarne la validità. Nel caso di risultato positivo viene popolato il *SecurityContext* con l'identità e i ruoli dell'utente che è stato autenticato.
- **SecurityConfiguration**: definisce le regole di autorizzazione per ciascun endpoint. Il sistema sviluppa un controllo di accesso sulla base dei ruoli dell'utente (**RBAC Role-based Access Control**), rispettando i tre attori (*PATIENT*, *MEDIC*, *ADMIN*). Una particolarità per gli endpoint relativi alle risorse del paziente, l'autorizzazione non si limita a verificare solo il possesso del ruolo *PATIENT*, ma svolge una valutazione dinamica riguardo l'identificativo del paziente in modo tale che corrisponda effettivamente all'utente autenticato (tramite il metodo **checkId()**), andando a realizzare un controllo di autorizzazione a livello di singola risorsa e non solo del ruolo generico. Per quanto riguarda ruoli con privilegi più ampi, come **Medico** e **Paziente** i due godono di condizioni di accesso alla piattaforma meno restrittiva, coerentemente con la necessità di supervisionare il paziente da parte del medico.

Questo sviluppo ci consente di garantire che un paziente possa consultare e modificare esclusivamente le proprie rilevazioni, che un medico possa consultare i dati di qualunque paziente ma creare o modificare prescrizioni terapeutiche solamente nell'ambito della propria attività clinica, e che l'amministratore di sistema mantenga la visibilità e controllo completo sull'anagrafica degli utenti, incluso il processo di verifica delle nuove registrazioni.

2.4.3 Model: repository e service layer

Il modello del dominio rispecchia la struttura richiesta nelle specifiche di progetto, andando a distinguere le varie entità. In particolare è stata effettuata una netta separazione tra:

- **Therapy**: rappresenta la prescrizione da parte del medico diabetologo per un determinato paziente (farmaco, numero di assunzioni giornaliere, quantità, eventuali indicazioni) soggetta inoltre ad uno stato di ciclo di vita (**TherapyStatus**, con valori quali *ACTIVE* e *SUSPENDED*) e la tracciabilità delle modifiche (campi **lastModifiedBy** e **lastModifiedAt**);

- **MedicineIntake**: rappresenta l'evento di assunzione registrato dal paziente (farmaco, quantità, data e ora), collegato tramite la relazione **@Many-ToOne** alla terapia di riferimento, e dotato di un attributo booleano (**matchesTherapy**) che calcola ciò che viene dichiarato dal paziente con quello prescritto;
- **Symptom, Pathology, ConcomitationTherapy**: sono tre entità di forma analoga che modellano sintomi, patologie e terapie concomitanti auto dichiarate dal paziente, indipendenti dal piano di cura assegnato dal medico diabetologo e che sono caratterizzate da un periodo di validità: la data di fine nulla (**endDate == null**) rappresenta che una condizione è ancora in corso, d'altro canto nel caso esistesse una data di fine indicherebbe che la condizione è cessata.

Come di consueto, ogni entità è affiancata da un'interfaccia Repository dedicata, che definisce (oltre alle operazioni CRUD ereditate) metodi di query specifici per il dominio, con il compito di recuperare le condizioni necessarie. La logica di business, concordata nel *Service*, è responsabile in particolare di:

- validazione dell'esistenza del paziente referenziato
- calcolo degli attributi derivati (aderenza alla terapia, superamento soglie glicemiche)
- orchestrare, nel caso del **NotificationService**, la generazione di notifiche a seguito di eventi applicativi (creazione di una rilevazione fuori dalla soglia) o di controlli pianificati (verifica giornaliera dell'aderenza terapeutica), impostata alle ore 20:00. Quest'ultima implementazione è stata fatta tramite un componente schedulato (**@Scheduled**) che itera periodicamente sull'insieme dei pazienti.

3 Test e validazione

Per testare il software sono stati usati diversi tipi di test, tra cui:

- Revisione del codice: il codice è stato revisionato parecchie volte per garantire che fosse stato scritto in modo che rispecchiasse le specifiche del progetto richieste.
- Verifica della consistenza: sono stati effettuati dei test per verificare che il software fosse coerente con le specifiche del progetto e che funzionasse, tramite testing di Spring Boot che permettono di testare le componenti. Questo test è automatizzato grazie all'API di JUnit.
- E' stato fatto gran uso, soprattutto nella parte iniziale, di Postman per testare tutti i controller del software, andando a verificare che le richieste HTTP restituissero i risultati attesi
- Test riguardante la sicurezza e la solidità del sistema di autenticazione.
- Il software è stato testato più volte dallo sviluppatore
- Il software è stato fatto testare anche da due utenti generici

3.1 Revisione del codice

La fase di revisione del codice di per sé è durata per tutto lo sviluppo del software, adottando controlli e maggiore attenzione per le parti che non funzionavano a livello di UI. Inoltre è stato controllato anche che il codice fosse stato scritto in modo chiaro e comprensibile e che rispettasse le specifiche del progetto. E' stato revisionato il codice per il controllo di eventuali bug o errori generali. Per poi terminare con una revisione finale di tutto il codice e anche la risposta visiva al termine del progetto.

3.2 Test dei service e dei repository

I test del codice, inseriti nella cartella test, sono degli unit test che verificano il corretto funzionamento dei Repository e dei Service. Per testare il software e' stata usata l'annotazione di Spring Boot `@DataJpaTest`, che permette di testare le interazioni con un database di test che e' salvato in memoria. Quello che segue e' un esempio di test per il repository del paziente, che verifica le operazioni CRUD (Create, Read, Update, Delete). Le funzioni con il tag `@Test` sono i test veri e propri e ad ogni test e' stato assegnato un ordine tramite l'annotazione `@Order`. Effettuiamo l'azione e verifichiamo che il risultato sia effettivamente quello atteso, usando le asserzioni di JUnit. La medesima cosa viene fatta per tutti i repository e i service del Model, mentre per il controller non sono stati fatti degli unit test, ma tutti i controller sono stati testati tramite Postman.

3.3 Test dei controller

Per testare le **API REST** del software sviluppato, ovvero ciò che i controller vanno ad esporre, è stato usato **Postman**. Si tratta di un tool gratuito che permette di inviare richieste HTTP e di visualizzare le risposte. Come sopra citato, questo test è stato fatto in gran parte durante l'implementazione iniziale del backend, permettendomi di simulare l'autenticazione degli utenti e anche iniettando dati fittizi riguardanti le assunzioni dei farmaci o delle terapie. Al momento dell'autenticazione di un utente, viene restituito un token JWT che viene usato per autenticare le richieste successive. Il token viene poi inserito nella cartella dell'ambiente di Postman ed usato per autenticare le richieste da parte di tutti i controller.

3.4 Test sicurezza

Nell'ambito delle attività di verifica del software, è stata effettuata, come attività aggiuntiva non espressamente richiesta nei requisiti, una valutazione preliminare della superficie d'attacco relativa a vulnerabilità di tipo **SQL Injection**. In particolare, sono stati usati pattern e payload rappresentativi di tecniche di SQL injection al fine di verificare la corretta validazione, sanificazione e gestione degli input da parte dell'applicazione e di accertare che gli stessi non potessero influenzare la costruzione o l'esecuzione delle query SQL.

L'attività è stata svolta autonomamente anche in considerazione del mio interesse personale nell'ambito della **Cybersecurity**, con l'obiettivo di approfondire ulteriormente gli aspetti di sicurezza dell'applicazione oltre ai test funzionali previsti. Ad esempio alcuni pattern usati sono:

- **Alterazione della condizione logica:** ' OR '1'='1
- **Chiusura della stringa e aggiunta di una condizione:** ' OR 1=1 –
- **Verifica di condizioni logiche false/vere:** ' AND 1=1 – e ' AND 1=2 –
- **Iniezioni di payload nella url tramite union:** ' UNION SELECT NULL –

3.5 Test manuale dello sviluppatore

In questa fase di test, che come citato poc'anzi è durata per tutto il tempo di sviluppo del software, mi sono impegnato a verificare manualmente che tutte le funzionalità implementate funzionassero come previsto, guardando anche con un occhio critico il lavoro svolto e impegnandomi a trovare una soluzione ottimale rispetto a quella implementata (sia a livello interfaccia utente sia per quanto riguarda la progettazione concettuale del software).

Riassumendo, i punti che sono stati testati con maggiore accuratezza sono stati:

- Verifica del login
- Verifica della registrazione
- Verifica che l'utente possa modificare i propri dati
- Verifica che le notifiche vengano create correttamente
- Verifica che le notifiche vengano cancellate correttamente
- Verifica dei vari inserimenti
- Verifica che le azioni da parte dell'amministratore funzionassero rispecchiando le specifiche

3.6 Test utenti generici

Infine per testare il software mi sono affidato a due utenti generici esterni, in modo tale da ricevere feedback sulle funzionalità implementate. In particolare entrambi gli utenti hanno effettuato una registrazione di un nuovo account e hanno provato ad usare il software come pazienti, replicando la stessa cosa fingendo di essere dei medici diabetologi. Il feedback che ho ricevuto è stato positivo e mi ha aiutato a migliorare anche la parte di interfaccia utente, in modo da rendere il software semplice, leggero ed intuitivo.